ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Простое веб-приложение. Верстка.»

Дисциплина: Разработка интернет-приложений

Выполнил(а): _____Головастикова П.Я._

Группа: ____ИБМ3-54Б_____

Проверил(а): _____

2026

Содержание

1. Цель и задачи лабораторной работы
2. Новые технологии и структура проекта
3. Сервер на Node.js и работа с npm
4. Коллекция mock-объектов и GET-запрос
5. Страница списка карточек и фильтрация
6. Страница подробной информации
7. Добавление и удаление карточек
8. Навигация и кнопка «Домой»
9. Проверка запросов во вкладке Network
10. Проверка работы и ограничения реализации
11. Заключение

1. Цель и задачи лабораторной работы

Цель лабораторной работы — познакомиться с Node.js и npm, а также расширить существующий интерфейс «Mercedes-AMG F1 — Центр производительности» страницей со списком карточек, фильтрацией, подробным просмотром и действиями над элементами коллекции.

Для достижения цели были поставлены следующие задачи:

- организовать запуск проекта через Node.js и команду npm start;
- создать коллекцию mock-объектов, описывающих болиды;
- реализовать получение коллекции с помощью GET-запроса;
- добавить страницу списка карточек с поиском и фильтрацией;
- реализовать отдельное представление с подробной информацией о выбранной карточке;
- добавить кнопку копирования первой карточки и кнопку удаления карточки;
- разместить кнопку «Домой» в шапке на обеих новых страницах;
- проверить получение HTML-документа и содержимое ответа во вкладке Network браузера.

2. Новые технологии и структура проекта

В третьей лабораторной работе существующий дизайн сайта не заменялся. Сохранена визуальная стилистика Mercedes-AMG PETRONAS: чёрный фон, тёмные панели, светлый текст, бирюзовые акценты, крупные заголовки и сеточное расположение элементов. Новые страницы используют те же принципы оформления и адаптивные правила, поэтому выглядят частью исходного приложения.

В отличие от первоначального варианта, который можно было открыть как отдельный HTML-файл, проект теперь запускается локальным HTTP-сервером. Сервер отдаёт HTML-страницу и предоставляет маршрут API для получения тестовых данных.

Основные файлы проекта:

- index.html — HTML-разметка, CSS-стили и JavaScript-логика интерфейса;
- server.js — сервер Node.js, который обрабатывает HTTP-запросы;
- package.json — метаданные проекта и команды запуска через npm;
- README.md — инструкция по запуску и проверке лабораторной работы.

3. Сервер на Node.js и работа с npm

Node.js позволяет выполнять JavaScript вне браузера. В данном проекте он используется для создания простого HTTP-сервера: сервер принимает запросы браузера и возвращает HTML-документ или данные коллекции. Для запуска проекта применяется npm — стандартный менеджер пакетов Node.js.

3.1. Файл package.json

В package.json указаны название проекта, версия, основной файл сервера и сценарии запуска. Команда start запускает сервер, а dev предназначена для аналогичного запуска в режиме разработки.

```
"scripts": {\n  "start": "node server.js",\n  "dev": "node server.js"\n}
```

В текущей версии дополнительные npm-пакеты не требуются: сервер использует встроенный модуль Node.js http. Поэтому установка зависимостей не обязательна. Для запуска достаточно открыть терминал в папке проекта и выполнить команду npm start.

```
npm start
```

После запуска сервер доступен по адресу http://127.0.0.1:3000. При обращении к корневому адресу сервер возвращает страницу index.html.

4. Коллекция mock-объектов и GET-запрос

Для выполнения задания использованы mock-объекты — заранее подготовленные тестовые записи, которые имитируют данные о болидах. Это позволяет изучать работу списка, фильтрации и карточек без подключения настоящей базы данных или внешнего сервиса.

Коллекция содержит несколько объектов с идентификатором и характеристиками болида. Сервер возвращает коллекцию по маршруту /api/cars в формате JSON. Браузер запрашивает данные, разбирает ответ и использует полученные объекты для построения карточек.

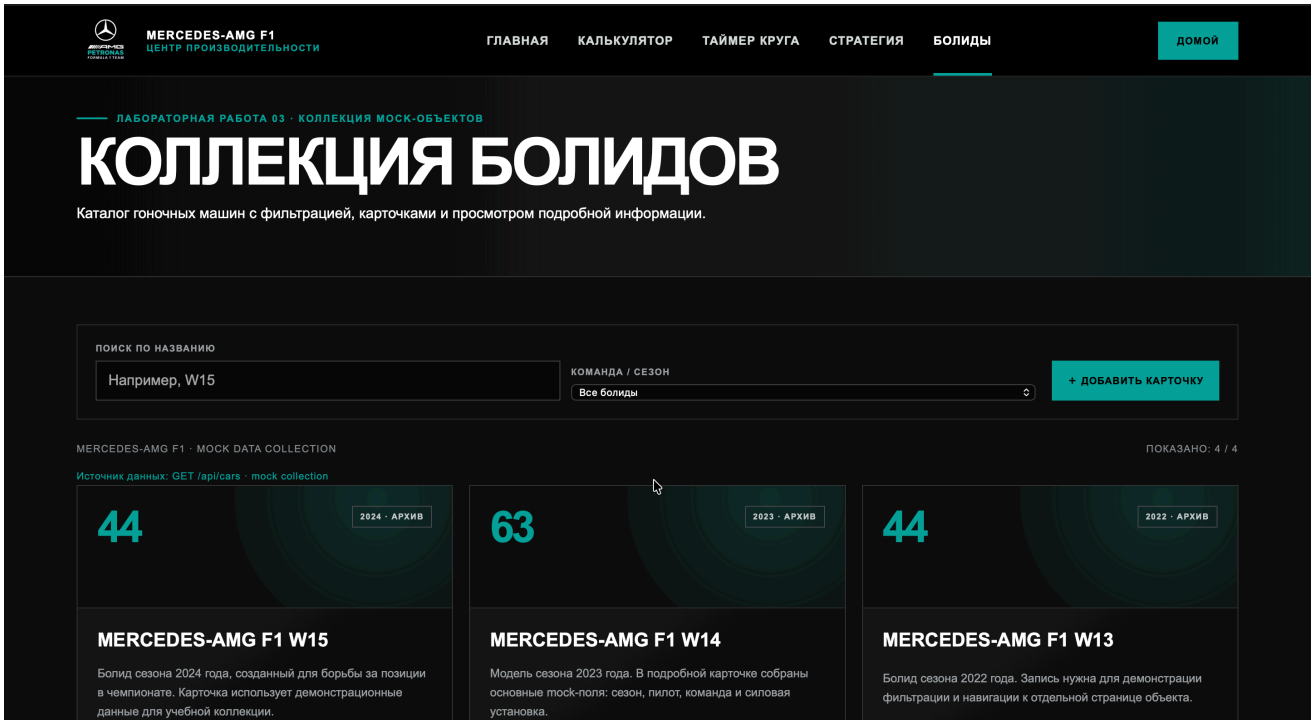
```
GET http://127.0.0.1:3000/api/cars
```

Метод GET применяется для чтения данных. В ответ на запрос сервер возвращает статус успешного выполнения и JSON-массив. Такой подход отделяет источник данных от представления: HTML отвечает за структуру интерфейса, JavaScript — за отображение и взаимодействие, а сервер — за выдачу данных.

4.1. Последовательность получения данных

1. Пользователь открывает сайт в браузере.
2. Браузер отправляет GET-запрос на корневой адрес сервера и получает HTML-документ.
3. JavaScript выполняет отдельный GET-запрос к /api/cars.
4. Сервер возвращает JSON-коллекцию mock-объектов.
5. Клиентский код использует массив объектов для построения карточек на странице.

5. Страница списка карточек и фильтрация



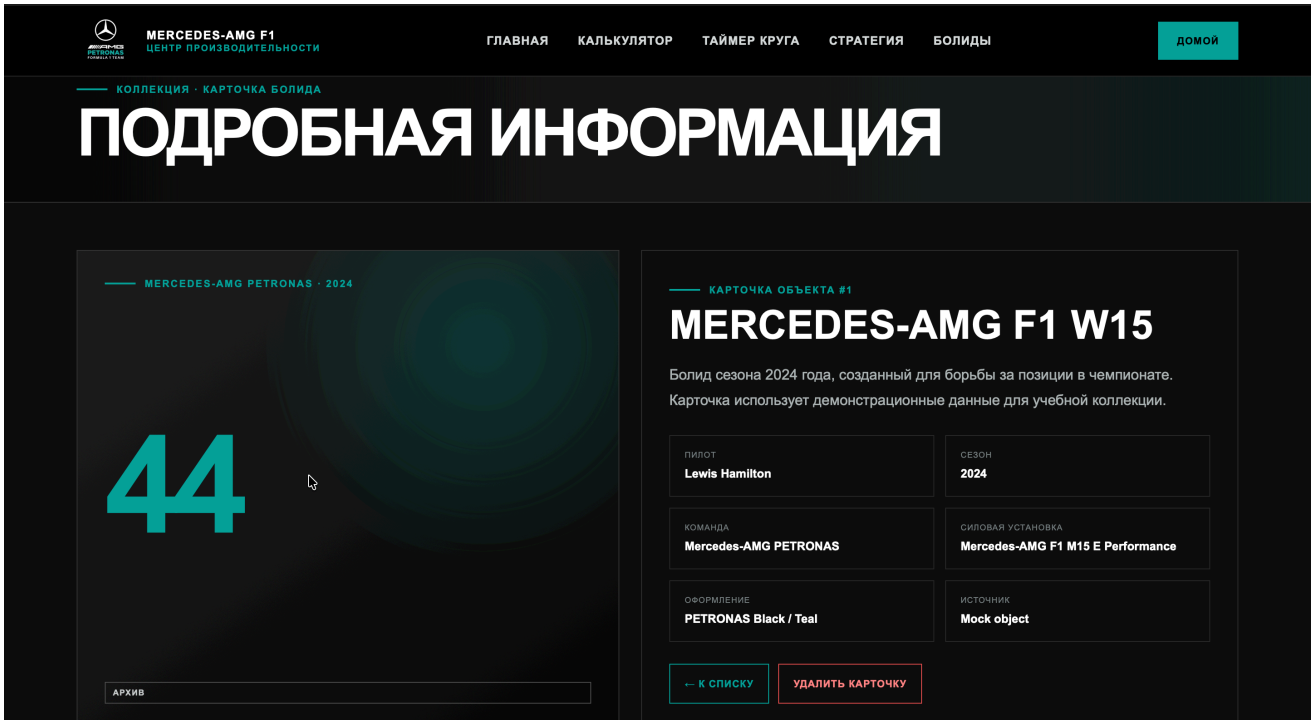
В проект добавлена отдельная страница «Болиды». На ней выводится коллекция карточек. Каждая карточка представляет отдельный объект и содержит основную информацию,

необходимую для быстрого ознакомления. Внешний вид карточек согласован с существующими тёмными панелями и бирюзовыми акцентами сайта.

Для удобства поиска добавлены элементы фильтрации. При вводе поискового текста или изменении условия фильтрации JavaScript проверяет данные объектов и обновляет отображаемый список. Фильтрация применяется к уже полученной mock-коллекции и не требует отдельного запроса к серверу при каждом изменении фильтра.

Карточки формируются динамически из массива данных. Такой способ удобнее ручного создания каждой карточки в HTML: при изменении коллекции список может перестроиться автоматически.

6. Страница подробной информации



Помимо списка, реализована страница подробного просмотра выбранного болида. При переходе к ней интерфейс определяет выбранную запись и выводит более подробные сведения. Таким образом, список предназначен для обзора и выбора объекта, а страница деталей — для изучения конкретной записи.

Переход к подробному просмотру связан с идентификатором объекта. Это важно, поскольку идентификатор позволяет отличить одну запись от другой даже при одинаковых или похожих названиях. Отдельная страница при этом сохраняет общую шапку и оформление сайта.

7. Добавление и удаление карточек

7.1. Добавление карточки

Для добавления предусмотрена кнопка, которая копирует первую карточку коллекции. Новая запись получает собственный идентификатор, чтобы не совпадать с исходным объектом. После добавления интерфейс обновляет список, и новая карточка становится доступна для просмотра и удаления.

Так как данные учебные, добавление выполняется в клиентском состоянии страницы. Оно демонстрирует логику работы с коллекцией, но не сохраняет изменения в постоянной базе данных. После перезапуска сервера или повторного получения исходной коллекции тестовые изменения могут исчезнуть.

7.2. Удаление карточки

На карточке предусмотрена кнопка удаления. При её нажатии JavaScript удаляет соответствующий объект из текущего массива по идентификатору и повторно отображает список. Использование идентификатора позволяет удалить выбранную запись, а не случайную карточку с похожим содержимым.

Добавление и удаление показывают базовые операции над коллекцией: создание новой записи на основе существующей и исключение выбранной записи из текущего набора данных.

8. Навигация и кнопка «Домой»

В шапке обеих новых страниц размещена кнопка «Домой». Она обеспечивает понятный путь обратно к главному экрану приложения независимо от того, находится ли пользователь в списке карточек или в подробном представлении.

Новая навигация встроена в существующую схему переключения экранов сайта. При этом сохранены исходные разделы — главная страница, калькулятор, таймер круга и демонстрационный расчёт гоночной стратегии. Новая функциональность дополняет проект, не меняя назначение ранее реализованных инструментов.

9. Проверка запросов во вкладке Network

Одно из требований лабораторной работы — показать GET-запрос, с помощью которого браузер получает HTML, и продемонстрировать содержимое ответа Response. Проверка выполняется во встроенных инструментах разработчика браузера.

9.1. Просмотр GET-запроса HTML

6. Запустить сервер командой npm start.
7. Открыть в браузере адрес http://127.0.0.1:3000.
8. Открыть инструменты разработчика клавишей F12 или через меню браузера.
9. Перейти на вкладку Network и перезагрузить страницу.
10. Выбрать запрос типа document, обычно с именем / или index.html.
11. На вкладке Headers показать Request Method: GET и успешный статус ответа, например 200 OK.
12. Открыть вкладку Response: в ней отображается HTML-код, который сервер передал браузеру.

Важно различать HTML-ответ и уже отрисованную страницу. В Response отображается исходный текст HTML-документа, полученный по сети; видимая страница является результатом обработки HTML, CSS и JavaScript браузером.

9.2. Просмотр ответа API

13. Не закрывая вкладку Network, перезагрузить страницу или перейти к списку болидов.
14. Найти запрос /api/cars; в зависимости от браузера его тип может отображаться как fetch или xhr.
15. Открыть вкладку Headers и показать метод GET и статус ответа.
16. Открыть Response или Preview и показать JSON-массив mock-объектов.

Таким образом, в Network можно показать два разных ответа: HTML-документ, который формирует основу страницы, и JSON-данные, которые используются для заполнения карточек. Это наглядно демонстрирует обмен данными между браузером и локальным сервером.

10. Проверка работы и ограничения реализации

10.1. Порядок проверки

17. Убедиться, что Node.js установлен, затем открыть терминал в папке проекта.
18. Выполнить npm start и открыть http://127.0.0.1:3000.
19. Проверить, что главная страница и прежние инструменты продолжают отображаться.
20. Открыть раздел «Болиды» и убедиться, что карточки загружаются из коллекции.
21. Ввести поисковый запрос и изменить фильтр, проверив обновление списка.
22. Открыть подробную информацию по одной из карточек.
23. Нажать кнопку добавления и проверить появление копии первой карточки.
24. Удалить одну из карточек и убедиться, что она исчезла из списка.
25. На обеих новых страницах проверить работу кнопки «Домой».
26. Во вкладке Network показать GET-запрос документа и его HTML Response, затем запрос /api/cars и JSON Response.

10.2. Ограничения

- Mock-объекты являются тестовыми данными и не представляют актуальную информацию о реальных болидах.
- Добавление и удаление демонстрируют операции с текущей коллекцией; постоянное хранение изменений в базе данных не реализовано.
- Локальный сервер предназначен для учебного запуска, а не для промышленного развёртывания.
- Фильтрация работает с полученным набором данных; отдельная серверная фильтрация не реализована.

11. Заключение

В ходе третьей лабораторной работы существующий проект «Mercedes-AMG F1 — Центр производительности» был расширен функциональностью списка карточек. В проект добавлены сервер Node.js, сценарии запуска через npm и маршрут GET /api/cars, возвращающий mock-коллекцию в формате JSON.

На стороне интерфейса реализованы страница списка болидов с фильтрацией, страница подробной информации, добавление копии первой карточки и удаление выбранной карточки. На обеих страницах предусмотрена кнопка «Домой». Новые элементы оформлены в прежней стилистике сайта, а исходные разделы приложения сохранены.

Дополнительно рассмотрен порядок проверки сетевого обмена во вкладке Network: GET-запроса, возвращающего HTML-документ, и GET-запроса к API, возвращающего JSON. Поставленная цель достигнута: лабораторная работа демонстрирует основы запуска JavaScript на сервере, получения данных по HTTP и динамического отображения коллекции на веб-странице.